Data Analytics using R and Python

Correlations, Association Rules & Decision Trees

Aayush Sharma

Department of CSE & IT

Kendall's Tau – What is it and Why do we Need it?

The Problem. Pearson's correlation measures *linear* relationships between two numeric variables and assumes the data is roughly normally distributed. But in many real situations the data is:

- **Ordinal** (ranks, preferences, Likert-scale responses like "strongly agree")
- **Non-linear** but monotonic (as X goes up, Y goes up, but not in a straight line)
- **Heavy-tailed or containing outliers** that distort Pearson's r

The Solution. **Kendall's Tau (τ)** is a non-parametric rank correlation coefficient proposed by Maurice Kendall in 1938. Instead of looking at actual values, it looks at the *order* of data points.

Key Intuition

If two variables X and Y are related, then whenever X increases, Y should also tend to increase (positive association) or decrease (negative association). Kendall's Tau counts **how often pairs of observations agree on this direction.**

Range: $\tau \in [-1, +1]$ where $+1$ = perfect agreement, 0 = no association, -1 = perfect reversal.

Concordant and Discordant Pairs – The Core Concept

Given two observations (x_i, y_i) and (x_j, y_j), we classify this pair as:

Concordant Pair

The pair moves in the *same* direction:

$$(x_i > x_j \text{ AND } y_i > y_j)$$

OR

$$(x_i < x_j \text{ AND } y_i < y_j)$$

Both variables agree on ordering.

Discordant Pair

The pair moves in *opposite* directions:

$$(x_i > x_j \text{ AND } y_i < y_j)$$

OR

$$(x_i < x_j \text{ AND } y_i > y_j)$$

The variables disagree on ordering.

Ties: If $x_i = x_j$ or $y_i = y_j$, the pair is neither concordant nor discordant. For tie-heavy data, use *Tau-b* or *Tau-c* variants.

Why this works: If X and Y are strongly related, almost every pair will be concordant. If they are unrelated, concordant and discordant pairs will be roughly equal in number.

Kendall's Tau – The Formula Explained

Standard Kendall's Tau (Tau-a)

$$\boxed{\tau = \frac{2S}{n(n-1)}} \quad \text{where } S = C - D$$

where C = concordant pairs, D = discordant pairs, $S = C - D$ = net score, n = number of observations.

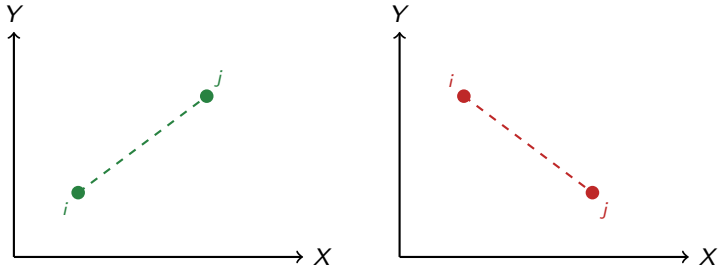
Breaking down the formula:

- $\frac{n(n-1)}{2} = \binom{n}{2}$ is the *total number* of pairs we can form.
- $S = C - D$ is the *net agreement*: positive if more concordant, negative if more discordant.
- Multiplying by 2 and dividing by $n(n-1)$ scales the answer to $[-1, +1]$.

Variants for ties:

- **Tau-b:** $\tau_b = \dfrac{C - D}{\sqrt{(C + D + T_x)(C + D + T_y)}}$ where T_x, T_y are tie counts. Used by R's `cor()`.
- **Tau-c:** Adjusts for different table sizes in contingency analysis.

Visualising Concordant vs Discordant



Reading this picture:

- **Left**: both points rise together – this pair *supports* a positive trend.
- **Right**: one rises while the other falls – this pair *opposes* a positive trend.
- If most pairs look like the left, τ is close to $+1$; if most look like the right, τ is close to -1.

Numerical Example: Two Judges Rank 5 Students

Scenario: Two judges independently ranked 5 students in a debate competition. We want to measure how much they agree.

Student	A	B	C	D	E
Judge 1 rank (x)	1	2	3	4	5
Judge 2 rank (y)	2	1	4	3	5

Step 1: Form all $\binom{5}{2} = 10$ pairs and check each:

Pair	$x_i < x_j$?	$y_i < y_j$?	Type	Reasoning
(A,B)	Yes (1¡2)	No (2¿1)	D	X up, Y down
(A,C)	Yes (1¡3)	Yes (2¡4)	C	Both up
(A,D)	Yes (1¡4)	Yes (2¡3)	C	Both up
(A,E)	Yes (1¡5)	Yes (2¡5)	C	Both up
(B,C)	Yes (2¡3)	Yes (1¡4)	C	Both up
(B,D)	Yes (2¡4)	Yes (1¡3)	C	Both up
(B,E)	Yes (2¡5)	Yes (1¡5)	C	Both up
(C,D)	Yes (3¡4)	No (4¿3)	D	X up, Y down
(C,E)	Yes (3¡5)	Yes (4¡5)	C	Both up
(D,E)	Yes (4¡5)	Yes (3¡5)	C	Both up

Numerical Example – Computing the Answer

Step 2: Count concordant and discordant pairs, compute S

$$C = 8, \quad D = 2, \quad S = C - D = 6, \quad n = 5$$

Step 3: Plug into the formula

$$\tau = \frac{2S}{n(n-1)} = \frac{2 \times 6}{5 \times 4} = \frac{12}{20} = \boxed{0.6}$$

Interpretation (How to Read This Result)

A value of $\tau = 0.6$ means:

- The two judges agree on the order of pairs 80% of the time (since $C/(C+D) = 8/10 = 0.8$).
- This is a **strong positive** rank correlation – the judges broadly agree, though not perfectly.
- For a thesis-defence committee, this would be considered acceptable inter-rater agreement; $\tau > 0.7$ is usually desired.

A Simpler Way to Count: The Column-Scan Method

The method you already know intuitively. Instead of examining every pair (i, j) separately, we use a slicker column-scan approach:

Step-by-Step Procedure

1. **Sort rows** by the X values in **ascending order** (rank 1 first, rank 2 next, ...).
2. **Look at the Y column** in the sorted order.
3. For each Y value, scan *all values below it* in the column and assign:
   - $+1$ (concordant) for every value *larger* than the current one.
   - -1 (discordant) for every value *smaller* than the current one.
4. Sum all the $+1$s and -1s to get $S = C - D$.
5. Apply the formula: $\tau = \dfrac{2S}{n(n-1)}$.

Why this works: Because the rows are already sorted by X (ascending), any later row automatically has a larger X value. So checking whether its Y is larger $(+)$ or smaller $(-)$ is exactly the same as checking concordance.

Column-Scan Method – Worked Example

Same data as before: 5 students, two judges. After sorting by Judge 1 rank (already 1–5):

Category	Judge 1 (X)	Judge 2 (Y)	Compare Y with all rows below
A	1	2	2 vs $\{1,4,3,5\}$: larger=$+3$, smaller=-1 $\Rightarrow$ $+3-1$
B	2	1	1 vs $\{4,3,5\}$: larger=$+3$, smaller=-0 $\Rightarrow$ $+3$
C	3	4	4 vs $\{3,5\}$: larger=$+1$, smaller=-1 $\Rightarrow$ $+1-1$
D	4	3	3 vs $\{5\}$: larger=$+1$, smaller=-0 $\Rightarrow$ $+1$
E	5	5	(last row, nothing below)

Tally:

$$C = 3+3+1+1 = 8, \quad D = 1+0+1+0 = 2$$

$$S = C - D = 8 - 2 = 6, \quad \tau = \frac{2S}{n(n-1)} = \frac{2\times 6}{5\times 4} = \frac{12}{20} = \boxed{0.6}$$

Same answer, less cognitive load! No need to list all 10 pairs explicitly – a single sweep down the Y

Kendall's Tau – Significance Testing

A value of $\tau = 0.6$ *looks* high, but is it statistically significant, or could it be due to chance?

Hypothesis test:

- H_0: $\tau = 0$ (no association between the two rankings)
- H_1: $\tau \neq 0$ (there is association)

For large n, under H_0:

$$z = \frac{3\tau\sqrt{n(n-1)}}{\sqrt{2(2n+5)}} \sim \mathcal{N}(0,1)$$

For small n (like our $n = 5$), exact p-values are computed from the permutation distribution of τ. R handles this automatically with `cor.test()`.

Rule of thumb for interpreting $|\tau|$:

- $0.00 - 0.10$: negligible
- $0.10 - 0.30$: weak
- $0.30 - 0.50$: moderate
- $0.50 - 0.70$: strong
- $0.70 - 1.00$: very strong

Kendall's Tau in R – Complete Walkthrough

```r
# ============================================================
# Kendall's Tau in R - step by step
# ============================================================

# 1. Enter the data (ranks given by two judges)
judge1 <- c(1, 2, 3, 4, 5)           # Judge 1's rankings
judge2 <- c(2, 1, 4, 3, 5)           # Judge 2's rankings

# 2. Compute Kendall's Tau using the built-in cor() function
#    method = "kendall" selects Tau-b (the tie-corrected version)
tau <- cor(judge1, judge2, method = "kendall")
print(paste("Kendall's Tau =", round(tau, 3)))   # -> 0.6

# 3. Test statistical significance
cor.test(judge1, judge2, method = "kendall")
#   T = 8, p-value = 0.2333, tau = 0.6

# 4. Visualise agreement
plot(judge1, judge2, pch = 19, col = "blue", cex = 2,
     main = "Rank Agreement Between Two Judges",
     xlab = "Judge 1 Rank", ylab = "Judge 2 Rank")
abline(0, 1, col = "grey", lty = 2)
text(judge1, judge2, labels = LETTERS[1:5], pos = 3)
```

Comparing Pearson, Spearman, and Kendall

	Pearson r	**Spearman** ρ	**Kendall** τ
What it measures	Linear relationship	Monotonic (rank)	Monotonic (pair agreement)
Data type	Continuous, normal	Ordinal/continuous	Ordinal/continuous
Outlier sensitivity	High	Low	Very low
Computation	Covariance/SDs	Pearson on ranks	Pair counting
Small samples	Unstable	Works	**Best**
Interpretation	r^2 = variance explained	Approximate Pearson on ranks	Probability of agreement minus disagreement

When to choose Kendall:

- Small sample size ($n < 30$)
- Presence of outliers
- You care about *direction of ordering* rather than magnitude
- Need a conservative, robust estimate

Kendall's Tau – Puzzle

★ **Puzzle:** A teacher ranked 6 students by *effort in class* and their *exam score*. The rankings are:

Student	A	B	C	D	E	F
Effort rank	1	2	3	4	5	6
Exam rank	1	3	2	5	4	6

Compute Kendall's τ and interpret the result.

Solution:

- Total pairs $= \binom{6}{2} = 15$, and $n = 6$.
- Go through all 15 pairs. Only *two* are discordant:
  - (B, C): effort $2 < 3$ but exam $3 > 2$
  - (D, E): effort $4 < 5$ but exam $5 > 4$
- So $D = 2$, $C = 13$, and $S = C - D = 11$.
- $\tau = \dfrac{2S}{n(n-1)} = \dfrac{2 \times 11}{6 \times 5} = \dfrac{22}{30} \approx \mathbf{0.733}$

Interpretation: Effort strongly predicts exam ranking. The 0.733 value would be considered "very strong" – only two pairs of students swapped ranks between the two measurements.

What is Affinity Analysis?

Definition

Affinity analysis is a data-mining technique that discovers meaningful **co-occurrence relationships** among items, events, or actions in large datasets. It answers the question:
"Which things tend to appear together?"

The most famous application: **Market Basket Analysis (MBA)** – analysing the contents of customer shopping baskets to find items frequently bought together.
Why this matters: Retailers and online platforms process millions of transactions daily. Hidden inside are patterns worth millions:

- If we know that X and Y are bought together, we can place them on nearby shelves, bundle them in promotions, or recommend Y when a user adds X to cart.
- Amazon's "Customers who bought this also bought..." is a direct application.
- Netflix recommendations, Spotify playlists, and fraud detection all use variants of affinity analysis.

Applications Across Industries

Retail & E-commerce

- Cross-selling ("frequently bought together")
- Store layout optimisation
- Catalog design, bundling, discounts
- Targeted coupons & loyalty offers

Web & Digital

- Web-page navigation patterns
- Clickstream analysis
- Recommender systems

Healthcare

- Co-occurring symptoms for diagnosis
- Drug interaction discovery
- Patient-treatment pattern analysis

Banking & Fraud

- Co-occurring transactions that suggest fraud
- Cross-product adoption (cards + loans + insurance)

Telecom

- Call-pattern analysis
- Service-bundle design

Education

- Courses taken together
- Error patterns in student answers

The "Beer and Diapers" Story

The legend: In the early 1990s, a retail chain analysed transaction data using MBA and found that **young fathers** often bought beer and diapers together on Friday evenings. The retailer supposedly placed these items on nearby aisles and sales rose.

Whether or not the exact story is true, it became the *origin myth* of market basket analysis for a reason – it perfectly captures the idea:

- The association is *surprising* (beer and diapers seem unrelated).
- It is *actionable* (change store layout, run joint promotions).
- It is *purely data-driven* – no one would have designed a survey to test "do beer-buyers buy diapers?"

The central question of MBA

Given a large transaction log, **which items (or sets of items) appear together more often than we would expect by chance**, and what do those co-occurrences tell us?

Setting Up the Transaction Database

A **transaction database** D is a collection of transactions, each of which is a set of *items* from a universe $I = \{i_1, i_2, \ldots, i_m\}$.

Running Example

TID	Items Purchased
T1	{Bread, Milk}
T2	{Bread, Diaper, Beer, Eggs}
T3	{Milk, Diaper, Beer, Coke}
T4	{Bread, Milk, Diaper, Beer}
T5	{Bread, Milk, Diaper, Coke}

Key vocabulary:

- **Itemset:** any subset of I, e.g., {Bread, Milk}.
- k-**itemset:** an itemset of size k.
- **Support count** $\sigma(X)$: number of transactions containing the itemset X.
- **Association rule:** an implication of the form $X \Rightarrow Y$ where $X, Y \subset I$ and $X \cap Y = \emptyset$.

From our example: $\sigma(\{\text{Bread, Milk}\}) = 3$ (it appears in T1, T4, T5).

Support – The First Pillar

Definition

$$\text{support}(X) = \frac{\sigma(X)}{|D|}$$

It is the *fraction of transactions* that contain the itemset X.

What it tells us: how **popular or common** the itemset is in the whole dataset.

Example computations (using our 5 transactions):

- support({Bread}) $= 4/5 = 0.80$ (in T1,T2,T4,T5)
- support({Diaper}) $= 4/5 = 0.80$ (in T2,T3,T4,T5)
- support({Diaper, Beer}) $= 3/5 = 0.60$ (in T2,T3,T4)

Why we need a minimum support threshold (minsup):

- Rules involving rare items are usually not statistically reliable.
- Counting millions of rare itemsets is computationally wasteful.
- We therefore prune any itemset below `minsup` (say, 0.05 or 5%) before looking for rules.

Confidence – The Second Pillar

Definition

For a rule $X \Rightarrow Y$:

$$\text{confidence}(X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)} = P(Y \mid X)$$

What it tells us: Among transactions that contain X, **what fraction also contain Y**? It is the conditional probability of Y given X.

Example (Diaper $\Rightarrow$ Beer):

- $\sigma(\{\text{Diaper}\}) = 4$, $\sigma(\{\text{Diaper, Beer}\}) = 3$
- Confidence $= 3/4 = 0.75 \Rightarrow 75\%$ of diaper-buyers also bought beer.

Pitfall: High confidence can mislead.

If 90% of all transactions contain milk, then almost any rule $X \Rightarrow$ Milk will have ≈ 0.9 confidence – but this tells us nothing useful. That's why we need...

Lift – The Third Pillar (Fixing Confidence's Blind Spot)

Definition

$$\text{lift}(X \Rightarrow Y) = \frac{\text{confidence}(X \Rightarrow Y)}{\text{support}(Y)} = \frac{P(Y \mid X)}{P(Y)}$$

What it tells us: How much **more likely** is Y to occur when X occurs, compared to Y's baseline frequency?

Interpretation:

- lift $= 1$: X and Y are *independent*; the rule is not informative.
- lift > 1: *positive* association – X makes Y more likely than by chance.
- lift < 1: *negative* association – X makes Y *less* likely.

Example (Diaper $\Rightarrow$ Beer): confidence $= 0.75$, support(Beer) $= 3/5 = 0.60$.

Lift $= 0.75/0.60 = \boxed{1.25}$

So diaper-buyers are **25% more likely** than random customers to buy beer – a real, positive association.

Worked Example – Applying all Three Pillars

Question: Evaluate the rule $\{\text{Diaper}\} \Rightarrow \{\text{Beer}\}$ using our 5 transactions.

Step 1: Count supports

- $\sigma(\{\text{Diaper}\}) = 4$ (T2, T3, T4, T5)
- $\sigma(\{\text{Beer}\}) = 3$ (T2, T3, T4)
- $\sigma(\{\text{Diaper, Beer}\}) = 3$ (T2, T3, T4)

Step 2: Compute each metric

$$\text{support}(D \Rightarrow B) = \frac{\sigma(D \cup B)}{|D|} = \frac{3}{5} = 0.60$$

$$\text{confidence}(D \Rightarrow B) = \frac{\sigma(D \cup B)}{\sigma(D)} = \frac{3}{4} = 0.75$$

$$\text{lift}(D \Rightarrow B) = \frac{0.75}{3/5} = 1.25$$

Conclusion: The rule appears in 60% of baskets, has 75% reliability, and lifts the probability of beer by 25%. This is a solid, actionable rule – worth placing beer near diapers.

Market Basket Puzzle

★ **Puzzle:** Consider 10 transactions over items $\{A, B, C\}$ with support counts:

$$\sigma(A) = 6,\ \sigma(B) = 5,\ \sigma(C) = 4$$
$$\sigma(AB) = 4,\ \sigma(AC) = 3,\ \sigma(BC) = 2,\ \sigma(ABC) = 2$$

Find support, confidence, and lift for the rule $\{A, B\} \Rightarrow \{C\}$. Is this a useful rule?

Solution step by step:

$$\text{support} = \frac{\sigma(A \cup B \cup C)}{10} = \frac{2}{10} = 0.20$$

$$\text{confidence} = \frac{\sigma(A, B, C)}{\sigma(A, B)} = \frac{2}{4} = 0.50$$

$$\text{lift} = \frac{\text{confidence}}{\text{support}(C)} = \frac{0.50}{4/10} = \frac{0.50}{0.40} = \mathbf{1.25}$$

Interpretation: 20% of baskets contain all three items; half of (A, B)-buyers also buy C; and they do

Why do we need the Apriori Algorithm?

The problem. Given a transaction database with m items, find all itemsets whose support is at least `minsup`. A naive enumeration would require checking all 2^m possible itemsets. For a modest supermarket with $m = 1000$ items, that is 2^{1000} – computationally impossible.

Two key observations that save us:

1. In practice, very few itemsets are frequent. Most combinations never appear.
2. Frequent itemsets have a beautiful *structural property* we can exploit.

The Apriori Principle (Downward Closure)

If an itemset is frequent, then **all its subsets** *must also be frequent.*

Contrapositive: if an itemset is infrequent, then **all its supersets** must also be infrequent – so we can prune them without counting.

Proposed by: Rakesh Agrawal and Ramakrishnan Srikant (IBM Research, 1994). It launched an entire subfield of data mining.

Why Downward Closure Works – Intuition and Proof

Intuition: If the set $\{A, B, C\}$ appears in at least 100 transactions, then $\{A, B\}$ must appear in *at least* those same 100 transactions (and possibly more). So the support of a subset is always $\geq$ support of the full set.

Short proof: Let $X \subseteq Y$. Any transaction T that contains Y must also contain X (since $X \subseteq Y \subseteq T$). Therefore

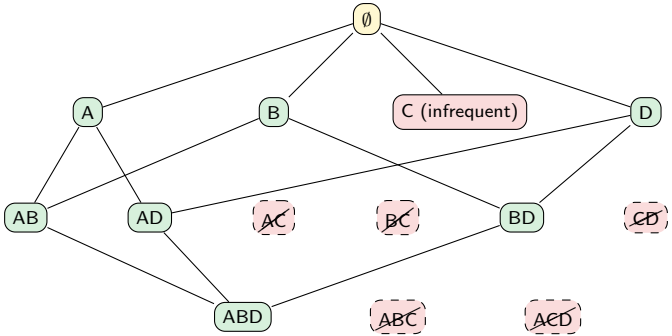
$$\sigma(X) \geq \sigma(Y) \quad \Rightarrow \quad \text{support}(X) \geq \text{support}(Y)$$

Consequence for pruning:

- If $\text{support}(\{C\}) < \texttt{minsup}$, then *any* itemset containing C – like $\{A, C\}, \{B, C\}, \{A, B, C\}$ – must also have support below `minsup`.
- We can throw all of them away **without ever counting them** in the database.

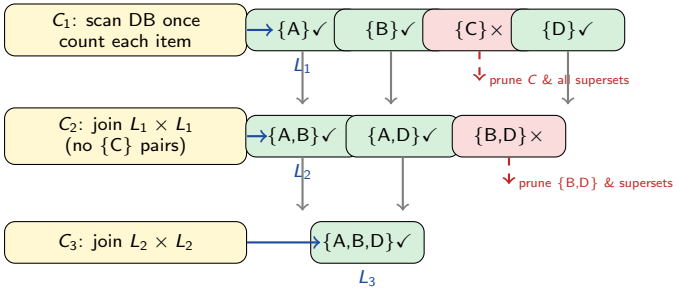
This is where Apriori gets its enormous speedup.

Visualising the Pruning on the Itemset Lattice



Reading the diagram: Once we discover that item C alone is not frequent, the downward-closure property tells us that every itemset containing C is automatically pruned – the whole red subtree disappears without a single database scan.

Apriori: How Pruning Saves Work – A Visual Walk-Through



Key insight: Once $\{C\}$ fails at level 1 and $\{B, D\}$ fails at level 2, **entire branches of the search tree are eliminated** – no database scans needed for those subtrees.

The Apriori Algorithm – Step by Step

Input: transaction database D, minimum support threshold `minsup`
Output: all frequent itemsets
Phase 1 – find L_1 (frequent 1-itemsets):

- Scan D once; count occurrences of each single item.
- Keep only those with support $\geq$ `minsup`.

Phase 2 – iterate for $k = 2, 3, \ldots$ until $L_{k-1} = \emptyset$:

1. **Join step:** Generate candidate set C_k by joining L_{k-1} with itself. Two $(k-1)$-itemsets are joined if they share a common $(k-2)$-prefix.
2. **Prune step:** For each candidate $c \in C_k$, check all its $(k-1)$-subsets. If any subset is not in L_{k-1}, remove c from C_k (downward closure).
3. **Count step:** Scan the database once, counting support of surviving candidates.
4. **Select:** $L_k = \{c \in C_k : \text{support}(c) \geq \texttt{minsup}\}$.

Return: $\bigcup_k L_k$ – the complete set of frequent itemsets.

Apriori – Numerical Example (Setup)

Transactions: ($|D| = 9$, set `minsup` $= 2$ meaning at least 22% of transactions)

TID	Items
T1	{1, 2, 5}
T2	{2, 4}
T3	{2, 3}
T4	{1, 2, 4}
T5	{1, 3}
T6	{2, 3}
T7	{1, 3}
T8	{1, 2, 3, 5}
T9	{1, 2, 3}

C_1 and L_1

Item	Count	Frequent?
{1}	6	✓
{2}	7	✓
{3}	6	✓
{4}	2	✓
{5}	2	✓

$L_1 = \{\{1\}, \{2\}, \{3\}, \{4\}, \{5\}\}$ – every 1-itemset clears minsup $= 2$.

Apriori – Generating L_2

Step 1 (join): Combine every pair from L_1 to get candidate 2-itemsets C_2.
No pruning needed yet (subsets of 2-itemsets are 1-itemsets, all of which are in L_1).
Step 2 (count): Scan D to count each candidate.

Itemset	Count	Frequent?	Reason
{1,2}	4	✓	T1,T4,T8,T9
{1,3}	4	✓	T5,T7,T8,T9
{1,4}	1	×	only T4
{1,5}	2	✓	T1,T8
{2,3}	4	✓	T3,T6,T8,T9
{2,4}	2	✓	T2,T4
{2,5}	2	✓	T1,T8
{3,4}	0	×	never appears
{3,5}	1	×	only T8
{4,5}	0	×	never appears

$L_2 = \{\{1,2\},\{1,3\},\{1,5\},\{2,3\},\{2,4\},\{2,5\}\}$

Apriori – Generating L_3 (Join + Prune)

Step 1 (join): Combine pairs in L_2 that share a common prefix of length 1.

- $\{1,2\} \bowtie \{1,3\} \rightarrow \{1,2,3\}$
- $\{1,2\} \bowtie \{1,5\} \rightarrow \{1,2,5\}$
- $\{1,3\} \bowtie \{1,5\} \rightarrow \{1,3,5\}$
- $\{2,3\} \bowtie \{2,4\} \rightarrow \{2,3,4\}$
- $\{2,3\} \bowtie \{2,5\} \rightarrow \{2,3,5\}$
- $\{2,4\} \bowtie \{2,5\} \rightarrow \{2,4,5\}$

Step 2 (prune): For each candidate, check that every 2-subset is in L_2.

- $\{1,3,5\}$: subset $\{3,5\} \notin L_2 \Rightarrow$ **prune**
- $\{2,3,4\}$: subset $\{3,4\} \notin L_2 \Rightarrow$ **prune**
- $\{2,3,5\}$: subset $\{3,5\} \notin L_2 \Rightarrow$ **prune**
- $\{2,4,5\}$: subset $\{4,5\} \notin L_2 \Rightarrow$ **prune**

Surviving $C_3 = \{\{1,2,3\},\{1,2,5\}\}$.

Apriori – Counting L_3 and Termination

Step 3 (count): Scan database for surviving candidates.

Itemset	Count	Frequent?
{1,2,3}	2 (T8, T9)	✓
{1,2,5}	2 (T1, T8)	✓

$L_3 = \{\{1, 2, 3\}, \{1, 2, 5\}\}$

Step 4 (try L_4): To form C_4, we would need to join two 3-itemsets in L_3 sharing a common 2-prefix. Here $\{1, 2, 3\}$ and $\{1, 2, 5\}$ share $\{1, 2\}$:

$$\{1, 2, 3\} \bowtie \{1, 2, 5\} \rightarrow \{1, 2, 3, 5\}$$

But its subset $\{3, 5\} \notin L_2$, so it is pruned. $C_4 = \emptyset$, and the algorithm **terminates**.

Final frequent itemsets: $L_1 \cup L_2 \cup L_3 = 13$ itemsets total.

Apriori in R – Using the arules Package

```r
# ============================================================
# Apriori in R
# ============================================================
install.packages("arules")      # only once
library(arules)

# Build transactions from a list
txn_list <- list(
  c("Bread","Milk"),
  c("Bread","Diaper","Beer","Eggs"),
  c("Milk","Diaper","Beer","Coke"),
  c("Bread","Milk","Diaper","Beer"),
  c("Bread","Milk","Diaper","Coke")
)
txn <- as(txn_list, "transactions")

# Mine frequent itemsets with minsup = 0.4 (40%)
freq_sets <- apriori(txn,
  parameter = list(supp = 0.4, target = "frequent itemsets"))
inspect(sort(freq_sets, by = "support"))

# Visualise item frequency
itemFrequencyPlot(txn, topN = 6, col = "steelblue")
```

Apriori – Complexity and Practical Tips

How many database scans?

At most $k_{\max} + 1$ scans, where $k_{\max}$ is the size of the longest frequent itemset.

How does Apriori's speed depend on parameters?

- Lower `minsup` $\Rightarrow$ more frequent itemsets $\Rightarrow$ slower, larger L_k.
- Longer transactions $\Rightarrow$ exponentially more candidate itemsets.
- More distinct items $\Rightarrow$ larger C_1 but pruning usually saves us.

Practical tips:

- Start with `minsup` high (e.g., 5%), then lower gradually.
- Set `maxlen` to cap itemset length and avoid runaway runs.
- Convert transactions to sparse matrices before running.
- For very large datasets, switch to FP-Growth (next section).

Apriori – Puzzle

★ **Puzzle:** In 8 transactions over items $\{A, B, C, D\}$, the first pass gives:

$$\sigma(A) = 6, \quad \sigma(B) = 5, \quad \sigma(C) = 3, \quad \sigma(D) = 1$$

With `minsup` $= 2$, list the candidate 2-itemsets (C_2) that actually need to be counted.

Solution reasoning:

1. $L_1 = \{A, B, C\}$. The itemset $\{D\}$ is dropped because $\sigma(D) = 1 < 2$.
2. By downward closure, any itemset containing D is also infrequent – we don't need to count $\{A, D\}, \{B, D\}, \{C, D\}$.
3. So C_2 includes only pairs built from L_1:

$$C_2 = \{\{A, B\}, \{A, C\}, \{B, C\}\}$$

Why this matters: Without downward closure we would have counted $\binom{4}{2} = 6$ pairs; Apriori counts only 3 – a 50% reduction. On real datasets with thousands of items, the savings become astronomical.

From Frequent Itemsets to Association Rules

Apriori (and FP-Growth) give us all frequent itemsets. But a retailer doesn't want itemsets – they want actionable rules like "*if customer buys bread and butter, they will also buy jam*".

Rule Generation – Two Steps

1. Find all frequent itemsets F with support $\geq$ `minsup`.
2. For every $I \in F$ with $|I| \geq 2$, generate every possible rule $A \Rightarrow (I \setminus A)$ where $\emptyset \neq A \subsetneq I$. Keep only those with confidence $\geq$ `minconf`.

How many rules can one 3-itemset produce?
From $I = \{A, B, C\}$ there are $2^3 - 2 = 6$ possible rules (excluding $\emptyset \Rightarrow I$ and $I \Rightarrow \emptyset$):

$$A \Rightarrow BC,\ B \Rightarrow AC,\ C \Rightarrow AB,\ AB \Rightarrow C,\ AC \Rightarrow B,\ BC \Rightarrow A$$

All six rules share the *same* support (it is support(I)) but have *different* confidences.

Efficient Rule Generation

Naive approach: for each frequent itemset, enumerate all $2^k - 2$ proper non-empty subsets and test each rule.

Smart pruning: there is a monotonicity property on rules too.

If a rule $A \Rightarrow (I \setminus A)$ has low confidence, then any rule with a smaller antecedent (and thus larger consequent) also has low confidence.

That is,

$$\text{conf}(A \Rightarrow B) \geq \text{conf}(A' \Rightarrow B') \quad \text{if } A' \subset A$$

Consequence: start with single-consequent rules like $ABC \Rightarrow D$; if confidence is low, there's no point trying $AB \Rightarrow CD$ or $A \Rightarrow BCD$.

This mirrors Apriori's pruning strategy, but applied to the rule lattice instead of the itemset lattice.

Rule Evaluation Metrics – the Full Toolkit

Metric	Formula	What it Captures
Support	$P(X \cap Y)$	How common is the pattern overall
Confidence	$P(Y \mid X)$	Reliability of the implication
Lift	$P(Y \mid X)/P(Y)$	Strength beyond chance
Leverage	$P(X \cap Y) - P(X)P(Y)$	Absolute excess co-occurrence
Conviction	$(1 - P(Y))/(1 - \text{conf}(X \Rightarrow Y))$	Implication strength; ∞ for certain rules
Jaccard	$P(X \cap Y)/P(X \cup Y)$	Set similarity between X and Y
All-conf.	$\min(P(Y \mid X), P(X \mid Y))$	The worst-case confidence

Why we need more than confidence:

- Confidence ignores $P(Y)$: a rule can have 99% confidence just because Y is everywhere.
- Lift corrects this but is symmetric in X and Y.
- Conviction is asymmetric and measures "how surprised should I be if the rule were wrong".
- Leverage gives the *amount* of extra joint probability – easier to interpret in business terms.

Worked Example – Rule Quality Comparison

From our 5-transaction dataset suppose we found the frequent itemset {Diaper, Beer} with support $3/5 = 0.60$. We derive two rules:

Rule	Supp	Conf	Lift	Comment
Diaper $\Rightarrow$ Beer	0.60	$3/4 = 0.75$	$0.75/0.60 = 1.25$	Strong rule
Beer $\Rightarrow$ Diaper	0.60	$3/3 = 1.00$	$1.00/0.80 = 1.25$	**Perfect implication**

Key insights:

- Both rules share the same support (0.60) and lift (1.25) – lift is a *symmetric* measure.
- Confidence differs: in our data, **every** beer purchase includes a diaper (conf $= 1.0$), but only 75% of diaper purchases include beer.
- A retailer would deploy Beer $\Rightarrow$ Diaper (higher confidence) as a checkout recommendation.

Lesson: Rules are *directional*. Never assume $X \Rightarrow Y$ and $Y \Rightarrow X$ are equivalent.

Association Rule Mining in R

```r
library(arules)
library(arulesViz)              # for rule visualisation

data("Groceries")               # 9835 real retail transactions
summary(Groceries)

# Mine rules with support >= 0.005 and confidence >= 0.5
rules <- apriori(Groceries,
                 parameter = list(supp = 0.005, conf = 0.5))

# Top 10 rules ranked by lift
top10 <- sort(rules, by = "lift")[1:10]
inspect(top10)

# Focus on rules ending in "whole milk"
milk_rules <- subset(rules, rhs %in% "whole milk")
inspect(sort(milk_rules, by = "confidence")[1:5])

# Scatter plot (support vs confidence, coloured by lift)
plot(rules, method = "scatter",
     measure = c("support", "confidence"), shading = "lift")

# Graph visualisation of top rules
plot(top10, method = "graph")
```

Association Rules – Puzzle

★ **Puzzle:** In a database of 1000 transactions: 600 contain *bread*, 750 contain *butter*, and 400 contain *both*. Compute support, confidence, and lift of the rule Bread ⇒ Butter. Is it actually a useful rule?

Solution:

$$\text{support} = 400/1000 = 0.40, \quad \text{confidence} = 400/600 \approx 0.667$$
$$P(\text{butter}) = 750/1000 = 0.75, \quad \text{lift} = 0.667/0.75 \approx \mathbf{0.89}$$

Interpretation – why this is a trap:

- The rule looks good: 40% support and 67% confidence.
- But lift < 1 shows a **negative association**. Bread buyers are *less* likely to buy butter than the average customer.
- The high confidence is entirely because butter is popular overall (75% baseline).

Lesson: Always check lift. Confidence alone can deceive.

The Limitations of Apriori

Apriori is elegant but has real-world performance problems:

1. **Multiple database scans.** One full scan per level k. For long frequent itemsets (length 10+), that's 10+ passes through potentially millions of transactions.
2. **Explosion of candidates.** If $|L_1| = 10\,000$, then $|C_2|$ can be $\binom{10000}{2} \approx 50$ million candidates before pruning. Counting each one is expensive.
3. **Redundant work.** Transactions are re-read from disk at every level.

FP-Growth (Han, Pei & Yin, 2000)

A radically different approach that mines frequent patterns **without generating candidates**. Key ideas:

- Compress the entire database into a compact tree data structure (the *FP-tree*).
- Mine the tree recursively using a divide-and-conquer strategy on conditional sub-trees.

Benefits: only 2 database scans; no candidate generation; typically 10× faster than Apriori on real data.

Building the FP-Tree – The Two Scans

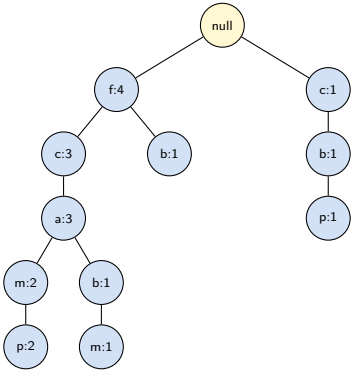
Scan 1: Count how often each item appears in the database. Drop items below `minsup`. Sort survivors in descending order of frequency. Call this order the *header table*.
Scan 2: For each transaction:

1. Keep only the frequent items.
2. Sort them in the header-table's frequency order.
3. Insert this sorted transaction into the FP-tree, sharing the prefix path with existing branches and incrementing counts along the shared path.

Why sorting by frequency?
Putting high-frequency items near the root maximises prefix sharing, giving a more compact tree. A small tree means less memory and faster mining.
Header table links: Every distinct item also has a linked list stitching together all its occurrences in the tree. This lets us find all paths containing an item in O(number-of-paths) time.

FP-Tree – Example Structure

Frequent items (after scan 1): $f:4, c:4, a:3, b:3, m:3, p:3$



Reading the tree:

- Node $f:4$ means "4 transactions passed through here containing f".
- The path root$\rightarrow f \rightarrow c \rightarrow a \rightarrow m \rightarrow p$ represents 2 transactions sharing this full prefix.
- Items with low frequency (short tails) occupy less tree real estate.

Mining the FP-Tree – Divide and Conquer

We mine patterns one item at a time, **bottom-up** in the header table (starting with the *least* frequent item).

For each item i:

1. Use the header-table links to collect *all prefix paths* ending in i (with their counts). This is the **conditional pattern base** of i.
2. Build a new, smaller FP-tree from these prefix paths – the **conditional FP-tree** for i.
3. Recursively mine the conditional FP-tree. Every frequent pattern found there, combined with i, is a frequent pattern of the original database.

Example – mining item p (support count 3)

- Prefix paths ending in p: $\langle f-c-a-m:2\rangle$ and $\langle c-b:1\rangle$.
- Conditional pattern base $= \{(f,c,a,m):2,\ (c,b):1\}$.
- In the conditional FP-tree, only c clears support $= 3$.
- Therefore the frequent pattern containing p is $\{c,p\}$ with support 3.

FP-Growth – Worked Mini-Example

Transactions: T1=$\{f, a, c, d, g, i, m, p\}$, T2=$\{a, b, c, f, l, m, o\}$, T3=$\{b, f, h, j, o\}$, T4=$\{b, c, k, s, p\}$, T5=$\{a, f, c, e, l, p, m, n\}$.
Set `minsup` $= 3$.
Scan 1 – item counts (keep only those with count ≥ 3):

Item	f	c	a	b	m	p
Count	4	4	3	3	3	3

Scan 2 – sort each transaction using order $f > c > a > b > m > p$:

- T1 $\rightarrow \{f, c, a, m, p\}$
- T2 $\rightarrow \{f, c, a, b, m\}$
- T3 $\rightarrow \{f, b\}$
- T4 $\rightarrow \{c, b, p\}$
- T5 $\rightarrow \{f, c, a, m, p\}$

Insert each into an empty FP-tree (sharing prefixes) to get the tree on the previous slide.
Mining the tree yields (among others): $\{f, c\} : 3,\ \{f, c, a\} : 3,\ \{f, c, a, m\} : 3,\ \{c, p\} : 3, \ldots$

FP-Growth-style Mining in R

```r
# ECLAT (equivalence-class) is closely related to FP-Growth
# and ships with the standard arules package.
library(arules)

data("Groceries")

# Find frequent itemsets via ECLAT
freq <- eclat(Groceries,
              parameter = list(supp = 0.01, maxlen = 5))
inspect(sort(freq, by = "support")[1:10])

# For an actual FP-Growth implementation, use rCBA or fim4r
# install.packages("rCBA")
# library(rCBA)
# fpg_rules <- fpgrowth(Groceries, support = 0.01,
#                       confidence = 0.5, consequent = NULL)
# inspect(head(fpg_rules, 10))
```

Note: base `arules` ships ECLAT, which is conceptually similar to FP-Growth (both avoid Apriori's candidate generation). For *native* FP-Growth from the C library by Christian Borgelt, use `rCBA` or `fim4r`.

Apriori vs FP-Growth – When to Use Which

Aspect	Apriori	FP-Growth
Search strategy	BFS, candidate-generation	DFS, pattern-fragment growth
Database scans	One per level $(k+1)$ total	Only 2
Candidate sets	Generated explicitly	**None**
Memory usage	Low per iteration	FP-tree in main memory
Typical speed	Baseline	$\sim 10\times$ **faster**
Parallelisation	Harder	Partition projections easily
Simplicity	Very easy to teach/debug	Trickier recursion
Best for	Teaching, small datasets	Production, huge datasets

Rule of thumb

Use **FP-Growth** in production on large, real-world datasets. Use **Apriori** when you want a clear, step-by-step algorithm for education or debugging, or when memory is very tight.

What is CART?

CART = **Classification And Regression Trees**, introduced by Leo Breiman, Jerome Friedman, Richard Olshen and Charles Stone in 1984. It is a foundational algorithm behind most modern tree-based methods (Random Forests, Gradient Boosting, XGBoost).

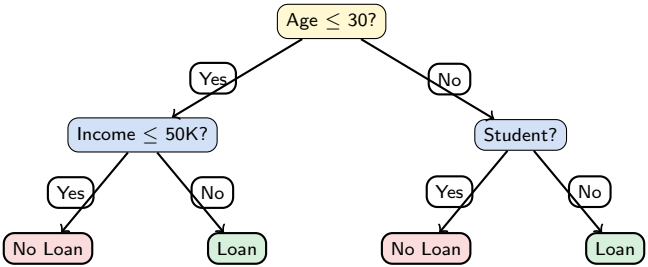
What CART does:

- Grows a **binary decision tree** by recursively splitting the data into two purer subsets at each node.
- Handles both classification (categorical target) and regression (numeric target) with a single unified framework.
- Can handle missing values via *surrogate splits*.
- Uses *cost-complexity pruning* to combat overfitting.

Why binary splits? Binary trees are:

- Easier to search (only two branches per node).
- More stable with small data (multi-way splits can over-fragment).
- A multi-way split can always be expressed as a series of binary splits.

A Simple CART Tree – What It Looks Like



How to read the tree:

- Each *internal node* asks a yes/no question about one feature (a "split").
- Each *leaf* gives the predicted class (or predicted value in regression).
- A new customer is classified by starting at the root and following the answers down to a leaf.

Splitting Criteria for Classification Trees

At every node CART must decide "which feature, and which split-value, gives the best binary partition?". "Best" means **maximum reduction in impurity**.

Gini Index (CART default)

$$\text{Gini}(D) = 1 - \sum_{k=1}^{K} p_k^2$$

- p_k = fraction of class k in node D
- Gini $= 0$: node is *pure* (all same class)
- Gini $= 1 - 1/K$: maximum impurity (uniform classes)
- Interpretation: probability of misclassifying a random sample if we label it by drawing at random from the node

Entropy (used by ID3 / C4.5)

$$H(D) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- $H = 0$: pure node
- $H = \log_2 K$: maximum impurity
- Interpretation: bits needed to encode the class of a random sample

Gini vs Entropy – practical difference: almost always produce very similar trees. Gini is slightly faster (no logarithm) and preferred by CART; entropy is preferred by ID3/C4.5.

Choosing the Best Split

Suppose a node D is partitioned by a candidate split into D_L and D_R.

Weighted impurity of the split

$$\text{Gini}_{\text{split}} = \frac{|D_L|}{|D|}\,\text{Gini}(D_L) + \frac{|D_R|}{|D|}\,\text{Gini}(D_R)$$

Gini gain (impurity reduction)

$$\Delta\text{Gini} = \text{Gini}(D) - \text{Gini}_{\text{split}}$$

CART algorithm at each node:

1. For every feature:
   - If numeric: consider every threshold between consecutive sorted values.
   - If categorical: consider every partition of categories into two groups.
2. Compute ΔGini for each candidate split.
3. Pick the split with the **largest** ΔGini.
4. Recurse on each child until a stopping rule fires (pure node, too few samples, or max depth).

CART Example – The Play Tennis Dataset

Day	Outlook	Temp	Humidity	Play?
1	Sunny	Hot	High	No
2	Sunny	Hot	High	No
3	Overcast	Hot	High	Yes
4	Rain	Mild	High	Yes
5	Rain	Cool	Normal	Yes
6	Rain	Cool	Normal	No
7	Overcast	Cool	Normal	Yes
8	Sunny	Mild	High	No
9	Sunny	Cool	Normal	Yes
10	Rain	Mild	Normal	Yes

Root node summary: 10 samples, 6 "Yes", 4 "No".

Compute root Gini:

$$\text{Gini}(D) = 1 - \left(\tfrac{6}{10}\right)^2 - \left(\tfrac{4}{10}\right)^2 = 1 - 0.36 - 0.16 = \mathbf{0.48}$$

Testing a Split on "Outlook"

Splitting on Outlook (3 values $\Rightarrow$ a 3-way split; CART turns this into the best binary partition, but conceptually we evaluate each branch):

Sunny (4 samples): {No, No, No, Yes} $\Rightarrow$ 1 Yes, 3 No

$$\text{Gini(Sunny)} = 1 - (1/4)^2 - (3/4)^2 = 1 - 0.0625 - 0.5625 = 0.375$$

Overcast (2 samples): {Yes, Yes} $\Rightarrow$ pure

$$\text{Gini(Overcast)} = 0$$

Rain (4 samples): {Yes, Yes, No, Yes} $\Rightarrow$ 3 Yes, 1 No

$$\text{Gini(Rain)} = 1 - (3/4)^2 - (1/4)^2 = 0.375$$

Weighted impurity and gain:

$$\text{Gini}_{\text{split}} = \tfrac{4}{10}(0.375) + \tfrac{2}{10}(0) + \tfrac{4}{10}(0.375) = 0.300$$

$$\Delta\text{Gini} = 0.48 - 0.30 = \boxed{0.18}$$

CART would now test every other feature and pick the one with the largest Δ. Typically Outlook wins the root in this dataset.

CART for Regression – Trees that Predict Numbers

For regression targets, CART replaces Gini with **variance / mean-squared error**:

Node MSE

$$\mathrm{MSE}(D) = \frac{1}{|D|} \sum_{i \in D} (y_i - \bar{y}_D)^2$$

where $\bar{y}_D$ is the mean of y values in the node.

Split selection: choose the split that maximises

$$\Delta = \mathrm{MSE}(D) - \left(\frac{|D_L|}{|D|} \mathrm{MSE}(D_L) + \frac{|D_R|}{|D|} \mathrm{MSE}(D_R) \right)$$

Prediction at a leaf: the *mean* of training y values that fall into that leaf.
Why MSE? Minimising weighted MSE is equivalent to maximising the *between-group variance* – the same Fisher-style criterion used in ANOVA. Leaves end up as homogeneous as possible in y.

Overfitting and Cost-Complexity Pruning

The overfitting problem: a fully grown tree can perfectly classify every training sample by creating very specific leaves. Such a tree memorises noise and performs poorly on new data.

Two defences:

1. **Pre-pruning (early stopping):** limit `maxdepth`, `minsplit`, `minbucket`, or require a minimum Δ at every split.
2. **Post-pruning (preferred by CART):** grow a full tree, then prune subtrees that fail a complexity test.

Cost-Complexity Criterion

$$R_\alpha(T) = R(T) + \alpha \cdot |T|$$

where $R(T)$ is training misclassification cost, $|T|$ is number of leaves, and $\alpha \geq 0$ is a tuning parameter.

How to choose α: k-fold cross-validation. For each α, compute the CV error and pick the α with lowest error (or within 1 SE, the "1-SE rule", for more conservative pruning).

CART in R – Classification on the Iris Dataset

```
install.packages(c("rpart","rpart.plot"))
library(rpart); library(rpart.plot)

# 70/30 train/test split
set.seed(42)
idx <- sample(seq_len(nrow(iris)), 0.7 * nrow(iris))
train <- iris[idx, ]; test <- iris[-idx, ]

# Fit a classification tree (CART uses Gini by default)
model <- rpart(Species ~ ., data = train, method = "class",
               parms   = list(split = "gini"),
               control = rpart.control(cp = 0.01, minsplit = 5))

# Visualise the tree
rpart.plot(model, type = 2, extra = 104, box.palette = "GnBu")

# Evaluate on the held-out test set
pred <- predict(model, test, type = "class")
cat("Test accuracy:", round(mean(pred == test$Species), 3), "\n")

# Cross-validated pruning using the cp table
best_cp <- model$cptable[which.min(model$cptable[,"xerror"]), "CP"]
pruned  <- prune(model, cp = best_cp)
```

CART in R – Regression on Air Quality Data

```r
library(rpart); library(rpart.plot)

# Built-in air-quality dataset; predict Ozone
data("airquality")
aq <- na.omit(airquality)

# Regression tree (method = "anova" for continuous targets)
reg_tree <- rpart(Ozone ~ Solar.R + Wind + Temp + Month,
                  data = aq, method = "anova",
                  control = rpart.control(cp = 0.01))

# Plot
rpart.plot(reg_tree, main = "Regression Tree for Ozone",
           box.palette = "RdYlGn")

# Training RMSE
pred <- predict(reg_tree, aq)
rmse <- sqrt(mean((aq$Ozone - pred)^2))
cat("Training RMSE:", round(rmse, 2), "\n")

# Variable importance ranking
print(reg_tree$variable.importance)
```

Strengths and Weaknesses of CART

Strengths

- Interpretable – you can literally read the decision path.
- Handles numeric *and* categorical features natively.
- Invariant to monotone transformations of features.
- No feature scaling needed.
- Handles missing values via surrogate splits.
- Automatic feature selection (unused features don't appear).

Weaknesses

- High variance – small changes in data can change the whole tree.
- Greedy at each split; not globally optimal.
- Struggles with linear relationships (needs stair-step approximations).
- Deep trees over-fit easily.
- Biased towards features with many levels (less of an issue in modern implementations).

Modern fixes: Random Forests average many CART trees on bootstrapped data; Gradient Boosting (XGBoost, LightGBM) trains trees sequentially to correct each other's errors.

CART – Puzzle

★ **Puzzle:** A node contains 20 samples: 16 positive, 4 negative.

- Compute Gini and entropy at the node.
- Consider a candidate split: Left = {12 pos, 0 neg}, Right = {4 pos, 4 neg}. Compute Gini gain.

Solution:

Node impurity:

$$\text{Gini}(D) = 1 - (16/20)^2 - (4/20)^2 = 1 - 0.64 - 0.04 = 0.32$$

$$H(D) = -0.8 \log_2 0.8 - 0.2 \log_2 0.2 \approx 0.722$$

Child impurities:

- $\text{Gini}(L) = 1 - 1^2 - 0^2 = 0$ (pure leaf)
- $\text{Gini}(R) = 1 - (0.5)^2 - (0.5)^2 = 0.5$ (max impurity)

Weighted split impurity and gain:

$$\text{Gini}_{\text{split}} = \tfrac{12}{20}(0) + \tfrac{8}{20}(0.5) = 0.20$$

$$\Delta\text{Gini} = 0.32 - 0.20 = \boxed{0.12} \quad \Rightarrow \text{ a useful split}$$

References & Further Reading

- Kendall, M.G. (1938). *A New Measure of Rank Correlation*. Biometrika, 30(1/2), 81–93.
- Agrawal, R., & Srikant, R. (1994). *Fast Algorithms for Mining Association Rules*. VLDB.
- Han, J., Pei, J., & Yin, Y. (2000). *Mining Frequent Patterns without Candidate Generation*. SIGMOD.
- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Tan, P.-N., Steinbach, M., & Kumar, V. (2018). *Introduction to Data Mining*. Pearson (2nd ed.).
- Hahsler, M., et al. `arules`: Mining Association Rules and Frequent Itemsets in R. CRAN.
- Therneau, T., & Atkinson, B. `rpart`: Recursive Partitioning and Regression Trees. CRAN.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R*. Springer.

Thank You!

Questions?

> "In God we trust. All others must bring data."
>
> — W. Edwards Deming

Aayush Sharma • Department of CSE & IT